

Projeto 5 - Leis de Kepler e o problema de três corpos.

13 de dezembro de 2023

Universidade de São Paulo - Instituto de Física de São Carlos.

Docente: José A. Hoyos

Tony Maciel Alexander. Número USP: 12624850

1 Tarefa 1 - Órbitas circulares: comparação entre os métodos de Euler-Cromer e Verlet.

Problema: Comparar o método de Euler-Cromer com o método de Verlet para a simulação das órbitas dos planetas do sistema solar (supondo órbita circular).

Para simular a órbita circular de um planeta, é preciso resolver a equação diferencial do seu movimento, que é dado pela segunda lei de Newton na equação (1):

$$m \frac{d^2 r}{dt^2} = -\frac{GMm}{r^3} \quad (1)$$

Onde M é a massa do Sol, e a direção de aplicação das forças em questão está na direção radial. Fazendo a seguinte mudança de variáveis: $\rho = \frac{1}{UA}r$ e $\tau = \frac{2\pi}{ano}t$, onde UA é uma unidade astronômica de comprimento. Segue imediatamente dessa mudança de variáveis e da equação (1), a equação (2):

$$\frac{d^2 \rho}{d\tau^2} = -\frac{GM}{(\rho UA)^3} \left(\frac{ano}{2\pi}\right)^2 = -\frac{1}{\rho^3} \quad (2)$$

Onde a direção das forças continua na direção radial. Na última igualdade da equação (2), foi usado o fato de que $GM = \left(\frac{2\pi}{ano}\right)^2 (UA)^3$. O que realmente ocorre, pois como a órbita é circular, a força centrípeta será:

$$\frac{mv^2}{r} = \frac{GMm}{r^2} \iff GM = \left(\frac{UA}{\left(\frac{ano}{2\pi}\right)}\right)^2 (UA) = \left(\frac{2\pi}{ano}\right)^2 (UA)^3 \quad \blacksquare$$

Para resolver a equação (2), será adotado por conveniência um sistema de coordenadas cartesianas. Logo, $\rho = \sqrt{\rho_x^2 + \rho_y^2}$ e a direção da força na equação (2) terá uma componente na direção x e na direção y. Além disso, como a equação (2) é de segunda ordem, é equivalente ao seguinte sistema de equações em cada direção (logo 'n' deve ser substituído por x e y):

$$\begin{cases} \dot{\rho}_n = v_n \\ \dot{v}_n = -\frac{\rho_n}{\rho^3} \end{cases} \quad (3)$$

Antes de usar o método de Euler-Cromer e o método de Verlet, é preciso obter as condições iniciais. Será suposto que um dado planeta começa no ponto $(x_0, y_0) = (a, 0)$, onde 'a' é o raio da órbita do planeta, com velocidade igual a $v_0 = \frac{1}{\sqrt{a}}$. A velocidade inicial foi obtida através da força centrípeta e da relação $GM = \left(\frac{2\pi}{ano}\right)^2 (UA)^3$:

$$\frac{dr}{dt} = \sqrt{\frac{GM}{r}} = \left(\frac{2\pi}{ano}\right) \frac{UA}{\sqrt{\frac{r}{UA}}} \iff \frac{d\rho}{d\tau} = \frac{1}{\sqrt{\rho}}$$

1.1 Métodos de Euler-Cromer e Verlet.

O método de Euler-Cromer será dado pela equação (3) em cada direção do movimento. O método de Verlet será dado pela equação (4) (que foi obtida pela série de Taylor da posição):

$$z_{i+1} = 2z_i - z_{i-1} + \ddot{z}_i(\Delta t)^2 \quad (4)$$

Adaptando para a equação (1) com a devida mudança de variáveis, segue a equação (5):

$$\rho_{i+1} = 2\rho_i - \rho_{i-1} - \frac{\rho_n}{\rho_i^3}(\Delta\tau)^2 \quad (5)$$

Onde 'n' deve ser substituído por 'x' e 'y'. Analogamente, a velocidade para esse método será dado subtraindo as séries de Taylor de $z(t + \Delta t)$ e $z(t - \Delta t)$, o que resulta na equação (6):

$$V_i = \frac{z(t + \Delta t) - z(t - \Delta t)}{2\Delta t} = \frac{z_{i+1} - z_{i-1}}{2\Delta t} + \mathcal{O}(\Delta t^3) \quad (6)$$

O código fonte da tarefa 1b está nas figuras (1) e (2). O resultado de ambas simulações apontam que os métodos são equivalentes para descreverem a órbita de planetas.

```

implicit none
real*8 , parameter :: dt = 1d-3
real*8 , parameter :: T_max = 1d2           ! end of simulation
real*8 , parameter :: r = 301d-1           ! distance of planet
real*8 :: Px0_ec, Vx0_ec, Px0_ver, Vx0_ver ! ec = Euler-Cromer
real*8 :: Py0_ec, Vy0_ec, Py0_ver, Vy0_ver ! ver = Verlet
real*8 :: Px1_ec, Vx1_ec, Px1_ver, Vx1_ver ! 0 = before
real*8 :: Py1_ec, Vy1_ec, Py1_ver, Vy1_ver ! 1 = after
real*8 :: Px2_ver, Py2_ver                 ! 2 = after after
real*8 :: T , P_ec, V_ec, P_ver, V_ver
! initial conditions
T = 0
px0_ec = r
py0_ec = 0
vx0_ec = 0
vy0_ec = 1d0/dsqrt(r)
P_ec = dsqrt(Px0_ec*Px0_ec + Py0_ec*Py0_ec)
V_ec = dsqrt(Vx0_ec*Vx0_ec + Vy0_ec*Vy0_ec)
Px0_ver = r
py0_ver = 0
vy0_ver = 1d0/dsqrt(r)
vx0_ver = 0
Px1_ver = Px0_ver + Vx0_ver*dt
Py1_ver = Py0_ver + Vy0_ver*dt
P_ver = dsqrt(Px0_ver*Px0_ver + Py0_ver*Py0_ver)
V_ver = dsqrt(Vx0_ver*Vx0_ver + Vy0_ver*Vy0_ver)
open(unit=1, file="tarefa-1b-saida-1.dat")
do while (T .lt. T_max)
    write(1,*) T, P_ec, V_ec, P_ver, V_ver
    ! Euler-Cromer
    P_ec = P_ec**(3d0)
    vx1_ec = vx0_ec - px0_ec/P_ec*dt
    px1_ec = px0_ec + vx1_ec*dt
    vy1_ec = vy0_ec - py0_ec/P_ec*dt
    py1_ec = py0_ec + vy1_ec*dt
    px0_ec = px1_ec
    py0_ec = py1_ec
    vx0_ec = vx1_ec
    vy0_ec = vy1_ec
    P_ec = dsqrt(Px0_ec*Px0_ec + Py0_ec*Py0_ec)
    V_ec = dsqrt(Vx0_ec*Vx0_ec + Vy0_ec*Vy0_ec)
    ! Verlet
    P_ver = P_ver**3d0
    px2_ver = 2*px1_ver - px0_ver - dt*dt*Px1_ver/(P_ver)
    vx1_ver = (px2_ver - px0_ver)/(2*dt)
    py2_ver = 2*py1_ver - py0_ver - dt*dt*Py1_ver/(P_ver)
    vy1_ver = (py2_ver - py0_ver)/(2*dt)
    vx0_ver = vx1_ver
    vy0_ver = vy1_ver
    px0_ver = px1_ver
    py0_ver = py1_ver

```

Figura 1: Código fonte da tarefa 1b (simula órbitas circulares de planetas pelos métodos de Euler-Cromer e Verlet).

```

do while (T .lt. T_max)
  write(1,*) T, P_ec, V_ec, P_ver, V_ver
  ! Euler-Cromer
  P_ec = P_ec**(3d0)
  vx1_ec = vx0_ec - px0_ec/P_ec*dt
  px1_ec = px0_ec + vx1_ec*dt
  vy1_ec = vy0_ec - py0_ec/P_ec*dt
  py1_ec = py0_ec + vy1_ec*dt
  px0_ec = px1_ec
  py0_ec = py1_ec
  vx0_ec = vx1_ec
  vy0_ec = vy1_ec
  P_ec = dsqrt(Px0_ec*Px0_ec + Py0_ec*Py0_ec)
  V_ec = dsqrt(Vx0_ec*Vx0_ec + Vy0_ec*Vy0_ec)
  ! Verlet
  P_ver = P_ver**3d0
  px2_ver = 2*px1_ver - px0_ver - dt*dt*Px1_ver/(P_ver)
  vx1_ver = (px2_ver - px0_ver)/(2*dt)
  py2_ver = 2*py1_ver - py0_ver - dt*dt*Py1_ver/(P_ver)
  vy1_ver = (py2_ver - py0_ver)/(2*dt)
  vx0_ver = vx1_ver
  vy0_ver = vy1_ver
  px0_ver = px1_ver
  py0_ver = py1_ver
  px1_ver = px2_ver
  py1_ver = py2_ver
  P_ver = dsqrt(Px2_ver*Px2_ver + Py2_ver*Py2_ver)
  V_ver = dsqrt(Vx1_ver*Vx1_ver + Vy1_ver*Vy1_ver)
  T = T + dt
end do
close(1)
call system("xmgrace -nxy tarefa-1b-saida-1.dat")
end

```

Figura 2: Outra parte de código fonte da tarefa 1b (simula órbitas circulares de planetas pelos métodos de Euler-Cromer e Verlet).

1.2 Verificação das órbitas circulares.

Para quantificar a órbita circular, será definido a grandeza $\delta = \frac{\rho_{max}}{\rho_{min}} - 1$. Em seguida, será simulado a órbita de um planeta pelos métodos de Euler-Cromer e Verlet a uma certa distância. Depois, para um tempo de simulação máximo de 500 unidades (quase 80 anos), à medida que δ não ultrapassa 10^{-3} (para cada método individualmente), $\Delta\tau$ é incrementado e a simulação é repetida. Ao final, tudo isso será repetido para diferentes raios de órbitas. O código fonte está nas figuras (3) e (4), e δ em função do incremento de tempo para um certo planeta com órbita de 10,5 UA está na figura (5).

Na figura (6), está uma tabela representando a quantidade $\Delta\tau_{max}/a^{3/2}$ para certas órbitas pelos métodos de Euler-Cromer e Verlet (onde 'a' é o semi-eixo maior, que nesse caso coincide com o raio da órbita circular). Essa quantidade está relacionada à terceira lei de Kepler pois um certo incremento de tempo para uma órbita pequena pode trazer uma precisão maior para órbitas maiores. Logo órbitas maiores devem ter $\Delta\tau_{max}$ maiores para um mesmo δ , e essa relação é regida exatamente pela lei de Kepler (pois para manter o erro constante, é como se o incremento de tempo fosse proporcional ao período daquela órbita).


```

        vx0_ec = vx1_ec
        vy0_ec = vy1_ec
        P_ec = dsqrt(Px0_ec*Px0_ec + Py0_ec*Py0_ec)

        ! Verlet
        P_ver = P_ver**3d0
        px2_ver = 2*px1_ver - px0_ver - dt*dt*Px1_ver/(P_ver)
        py2_ver = 2*py1_ver - py0_ver - dt*dt*Py1_ver/(P_ver)
        px0_ver = px1_ver
        py0_ver = py1_ver
        px1_ver = px2_ver
        py1_ver = py2_ver
        P_ver = dsqrt(Px2_ver*Px2_ver + Py2_ver*Py2_ver)
        T = T + dt
        if (P_ec .gt. Pmax_ec) then
            Pmax_ec = P_ec
        end if
        if (P_ec .lt. Pmin_ec) then
            Pmin_Ec = P_ec
        end if
        if (P_ver .gt. Pmax_ver) then
            Pmax_ver = P_ver
        end if
        if (P_ver .lt. Pmin_ver) then
            Pmin_ver = P_ver
        end if
    end do
    write(1,*) dt, Pmax_ec/Pmin_ec - 1, Pmax_ver/Pmin_ver - 1

    if (Pmax_ec/Pmin_ec - 1 .ge. 1d-3) then
        if (x .eqv. .false.) then
            x = .true.
            del1 = dt
        end if
    end if
    if (Pmax_ver/Pmin_ver - 1 .ge. 1d-3) then
        if (y .eqv. .false.) then
            y = .true.
            del2 = dt
        end if
    end if
    if ((x .eqv. .true.) .and. (y .eqv. .true.)) then
        goto 2
    end if
    dt = dt + r*1d-5
end do
! if delta t max still hasnt been found, repeat simulations...
goto 1
close(1)
print*,r,del1,del2,del1/r**(15d-1),del2/r**(15d-1)
r = r + 5d-1
end do
call system("xmgrace -nxy tarefa-1c-saida-1.dat") ! for last r
end

```

Figura 4: Código fonte da tarefa 1c (Achar Δt_{max} de órbitas circulares pelos métodos de Euler-Cromer e Verlet).

Error in orbit simulation versus step size

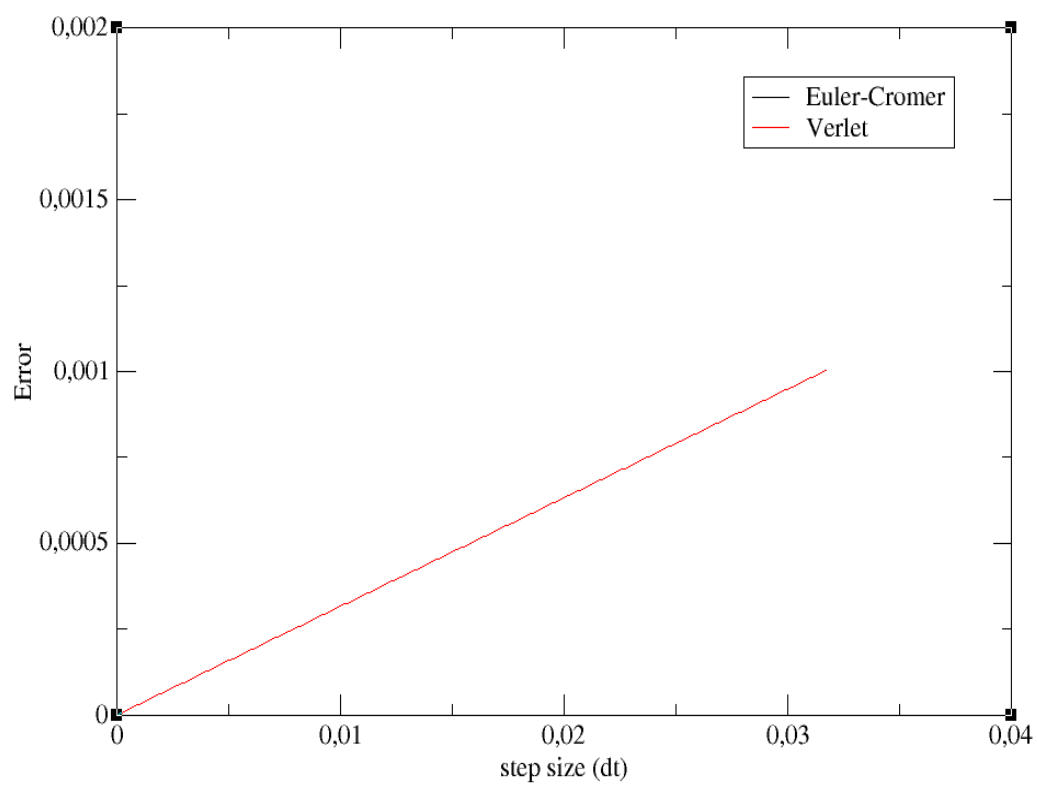


Figura 5: Erro na simulação de órbitas circulares pelos métodos de Euler-Cromer e Verlet em função do incremento de tempo utilizado.

Radius	dt_MAX with EC	dt_MAX with Ver	dt/a**(3/2) for EC	dt/a**(3/2) for Ver
0.5000000000000000	3.550000000000061E-004	3.550000000000061E-004	1.0040916292848991E-003	1.0040916292848991E-003
1.0000000000000000	1.000000000000020E-003	1.000000000000020E-003	1.000000000000020E-003	1.000000000000020E-003
1.5000000000000000	1.845000000000033E-003	1.845000000000033E-003	1.0042907945411049E-003	1.0042907945411049E-003
2.0000000000000000	2.840000000000061E-003	2.840000000000061E-003	1.0040916292848995E-003	1.0040916292848995E-003
2.5000000000000000	3.974999999999898E-003	3.974999999999898E-003	1.0056042959335420E-003	1.0056042959335420E-003
3.0000000000000000	5.219999999999920E-003	5.219999999999920E-003	1.0045894683899472E-003	1.0045894683899472E-003
3.5000000000000000	6.545000000000187E-003	6.545000000000187E-003	9.9955704475247005E-004	9.9955704475247005E-004
4.0000000000000000	8.000000000000175E-003	8.000000000000175E-003	1.000000000000022E-003	1.000000000000022E-003
4.5000000000000000	9.584999999999824E-003	9.584999999999824E-003	1.0040916292848958E-003	1.0040916292848958E-003
5.0000000000000000	1.119999999999960E-002	1.119999999999960E-002	1.0017584539199023E-003	1.0017584539199023E-003
5.5000000000000000	1.292499999999975E-002	1.292499999999975E-002	1.0020433668713670E-003	1.0020433668713670E-003
6.0000000000000000	1.469999999999934E-002	1.469999999999934E-002	1.0002083116364600E-003	1.0002083116364600E-003
6.5000000000000000	1.657500000000079E-002	1.657500000000079E-002	1.0001922892047433E-003	1.0001922892047433E-003
7.0000000000000000	1.855000000000084E-002	1.855000000000084E-002	1.0016058534744565E-003	1.0016058534744565E-003
7.5000000000000000	2.054999999999978E-002	2.054999999999978E-002	1.0005065383761023E-003	1.0005065383761023E-003
8.0000000000000000	2.264000000000052E-002	2.264000000000052E-002	1.0005560953789670E-003	1.0005560953789670E-003
8.5000000000000000	2.482000000000189E-002	2.482000000000189E-002	1.001551737322592E-003	1.001551737322592E-003
9.0000000000000000	2.699999999999948E-002	2.699999999999948E-002	9.999999999999807E-004	9.999999999999807E-004
9.5000000000000000	2.935500000000145E-002	2.935500000000145E-002	1.0025283825881174E-003	1.0025283825881174E-003
10.0000000000000000	3.169999999999881E-002	3.169999999999881E-002	1.0024420182733725E-003	1.0024420182733725E-003

Figura 6: Raios de órbitas circulares e o máximo incremento de tempo que garante um erro de 10^{-3} pelos métodos de Euler-Cromer e Verlet. Nas duas últimas colunas, está a terceira lei de Kepler (espera-se que os valores sejam constantes).

1.3 Comparação das energias.

A energia de um planeta será dado pela soma de sua energia potencial com sua energia cinética. Ou seja:

$$E = \frac{mv^2}{2} - \frac{GMm}{r}$$

Realizando as substituições $\rho = \frac{1}{UA}r$ e $\tau = \frac{2\pi}{ano}t$, a nova fórmula da energia total será dada pela equação (7):

$$E = \frac{m \left(\frac{UA}{\left(\frac{ano}{2\pi} \right)} \right)^2}{2} \left(\frac{d\rho}{d\tau} \right)^2 - \frac{\left(\frac{2\pi}{ano} \right)^2 (UA)^3 m}{(UA)\rho} = \left(\frac{m\rho^2}{2} - \frac{m}{\rho} \right) \left(\frac{2\pi UA}{ano} \right)^2 \quad (7)$$

O cálculo da energia de um planeta com órbita circular pelos métodos de Euler-Cromer e Verlet é dado simplesmente simulando a órbita e, em seguida, para cada iteração, substituindo a velocidade e posição na equação (7). O código fonte não está apresentado por ser muito parecido com as figuras (1) e (2) (a

única mudança está na primeira linha após o "do while", que escreve no arquivo de saída a energia para cada método pela equação (7)). O resultado obtido está representado na figura (7). Com isso, pode ser concluído que ambos métodos apresentam energias extremamente parecidos, sendo o método de Euler-Cromer ligeiramente melhor (a máxima diferença de energia ao longo de uma órbita é da ordem de 10^{-6}).

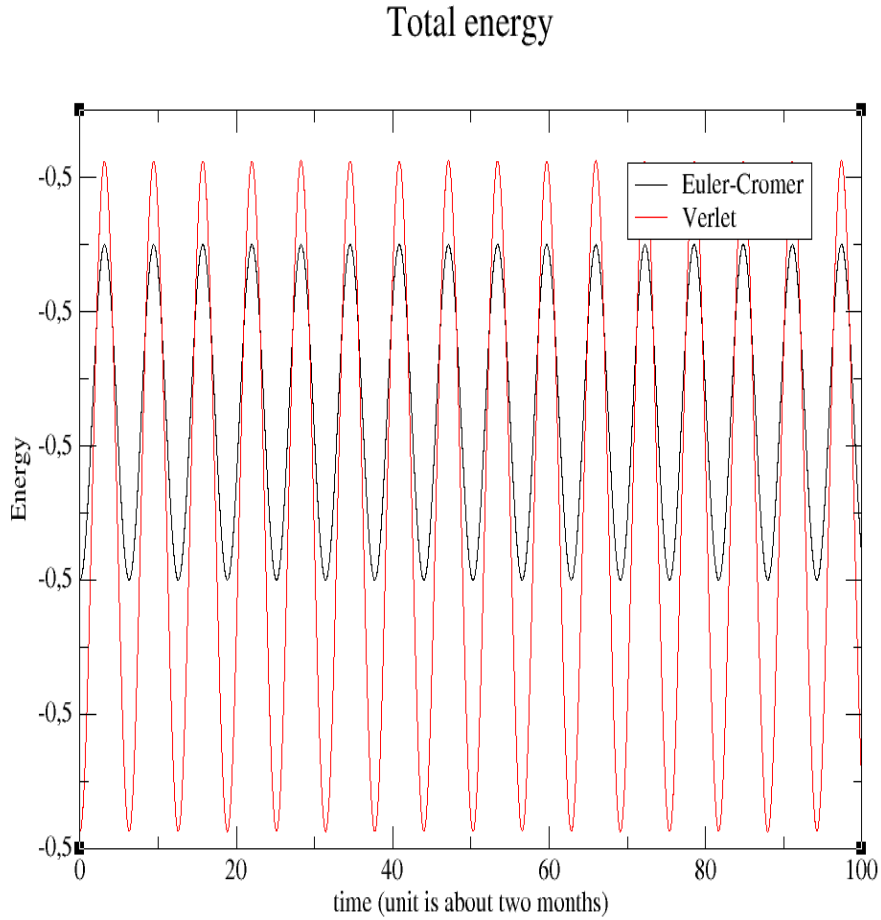


Figura 7: Energia de uma órbita circular para os métodos de Euler-Cromer e Verlet.

2 Tarefa 2 - O problema de dois corpos: órbitas limitadas.

Problema: Verificar as leis de Kepler para a simulação de órbitas (fechadas) dos planetas do sistema solar com o método de Verlet.

Primeiramente, a órbita de um planeta será fechada se a sua velocidade for menor que a velocidade limite (ou de escape) , i.e., a menor velocidade que um

planeta pode ter para escapar da sua órbita e parar "no infinito". Matematicamente, pela conservação de energia, a maior velocidade que um planeta pode ter para ter uma órbita fechada é:

$$\frac{mv_e^2}{2} - \frac{GMm}{r} = \frac{m(0)^2}{2} - \lim_{R \rightarrow \infty} \frac{GMm}{R} \iff v_e = \sqrt{\frac{2GM}{r}}$$

Usando as unidades adimensionais, a velocidade de escape é dada pela fórmula (8):

$$v_e = \sqrt{\frac{2(UA)^3 4\pi^2}{ano^2(UA)\rho}} = \frac{(UA)2\pi}{ano} \sqrt{\frac{2}{\rho}} \quad (8)$$

Em seguida, para verificar as leis de Kepler, será sempre mantido (para facilitar as contas) uma velocidade inicial somente no eixo y positivo. Assim, a elipse formada não será inclinada. Além disso, para calcular o semi-eixo maior e menor da órbita, será simulado a órbita e registrado as posições de Y máximo e mínimo, assim como as posições de X máximo e mínimo. Depois, será tomado a distância entre esses pontos. O semi-eixo maior será metade do maior dessas distâncias e o semi-eixo menor será metade do que sobrou.

O código fonte está nas figuras (8) e (9). Os resultados para algumas distâncias e velocidades iniciais estão nas figuras (10), (11) e (12). Não foi escolhido velocidades acima daquela na equação (8), para ter órbitas fechadas (embora a figura (12) chega perto, só pra ilustrar a validade do código fonte para quaisquer órbitas).

Em geral, está tudo de acordo com o esperado. Vale a pena notar que as oscilações da distância do planeta aos focos é devido à escolha do passo temporal (dt), pois se este for diminuído, essas oscilações também diminuem.

```

implicit none
real*8, parameter :: dt = 1d-4
real*8, parameter :: two_dt = 2d0*dt
real*8, parameter :: T_max = 8d2 ! chosen for xmgrace's autoscale
real*8, parameter :: initial_distance = 5d-1 ! change this
real*8, parameter :: initial_velo = 1d0 ! change this
real*8 :: Px0, Px1, Px2, Py0, Py1, Py2
real*8 :: P, V, T, Vx0, Vx1, Vy0, Vy1, temp1
real*8 :: x_min, y_max, y_min, period, f2, a, b
integer :: temp2 ! to count oscillations for period
T = 0
period = 0
Px0 = initial_distance
Py0 = 0
Vx0 = 0
Vy0 = initial_velo
Px1 = Px0 + Vx0*dt
Py1 = Py0 + Vy0*dt
P = norm(Px0,Py0)
V = norm(Vx0,Vy0)
y_max = Py1
y_min = Py1
x_min = px1
temp2 = 0
do while (T .lt. T_max)
! initial simulation to find y_max, period and f2
temp1 = dt*dt/P**3d0
Px2 = 2d0*Px1 - Px0 - Px1*temp1
Py2 = 2d0*Py1 - Py0 - Py1*temp1
Px0 = Px1
Py0 = Py1
Px1 = Px2
Py1 = Py2
P = norm(Px2,Py2)
T = T + dt
if (y_max .lt. Py1) then
y_max = Py1
end if
if (y_min .gt. Py1) then
y_min = Py1
end if
if (x_min .gt. Px1) then
x_min = Px1
end if
if (Py1*Py0 .le. 0d0) then
temp2 = temp2 + 1
period = T
end if
end do

period = period/temp2*2
b = (dabs(y_max) + dabs(y_min)) / 2
a = (dabs(x_min) + initial_distance) / 2

```

Figura 8: Código fonte da tarefa 2a (Verificação das leis de Kepler para órbitas fechadas).

```

period = period/temp2*2
b = (dabs(y_max) + dabs(y_min)) / 2
a = (dabs(x_min) + initial_distance) / 2

if (b .gt. a) then ! swap
  temp1 = a
  a = b
  b = temp1
end if
f2 = 2*dsqrt(a**2 - b**2)

if (a .gt. initial_distance) then ! find where f2 is
  f2 = -f2
end if

print*, "Kepler's constant from third law: ",
&(period**2)/(a**3)

open(unit=1, file="tarefa-2a-saida-2.dat")
T = 0
Px0 = initial_distance
Py0 = 0
Vx0 = 0
Vy0 = initial_velo
Px1 = Px0 + Vx0*dt
Py1 = Py0 + Vy0*dt
P = norm(Px0,Py0)
V = norm(Vx0,Vy0)
do while (T .lt. T_max)
  temp1 = dt*dt/P**3d0
  Px2 = 2d0*Px1 - Px0 - Px1*temp1
  Vx1 = (Px2 - Px0)/(two_dt)
  Py2 = 2d0*Py1 - Py0 - Py1*temp1
  Vy1 = (Py2 - Py0)/(two_dt)
  Vx0 = Vx1
  Vy0 = Vy1
  Px0 = Px1
  Py0 = Py1
  Px1 = Px2
  Py1 = Py2
  P = norm(Px2,Py2)
  T = T + dt
  write(1,*) T , (Px1*Vy1 - Py1*Vx1), (P + norm(Px2 - f2,Py2)),2*a
end do
close(1)
call system("xmgrace -nxy tarefa-2a-saida-2.dat")
contains
function norm(a,b)
  real*8 , intent(in) :: a,b
  real*8 :: norm
  norm = dsqrt(a*a + b*b)
end function
end

```

Figura 9: Código fonte da tarefa 2a (Verificação das leis de Kepler para órbitas fechadas).

2.1 Primeira lei.

Para verificar a primeira lei de Kepler, será suposto que o Sol está em um dos focos do planeta em questão e o outro foco terá sua distância calculada a partir dos semi-eixos. Se a soma das distâncias do planeta ao longo da sua órbita com o Sol e o outro foco for constante, então por definição o planeta tem uma órbita elíptica. Além disso, por hipótese o Sol também estará em um dos focos dessa órbita, o que completa a prova da primeira lei de Kepler (na verdade os focos são os únicos pontos dentro da elipse que satisfazem essa propriedade, então a escolha do Sol como sendo um foco está justificado).

A distância entre os dois focos de uma elipse é duas vezes o produto da excentricidade da órbita pelo semi-eixo maior. Como nesse caso, um foco está no centro da elipse, a soma da distância do planeta com os dois focos será dado por:

$$\sqrt{P_x^2 + P_y^2} + \sqrt{(P_x \pm 2\sqrt{a^2 - b^2})^2 + P_y^2} = 2a$$

Onde P_x e P_y são as componentes x e y do vetor posição. A razão de ter um mais ou menos na posição do segundo foco está relacionado com a velocidade inicial do planeta: se sua velocidade for muito grande, pode ser que a posição inicial não seja mais o ponto de maior afastamento com o Sol, o que levaria o foco a estar na direita do Sol (logo seria subtraído com P_x). Por definição, essa soma de distâncias tem que ser igual ao dobro do semi-eixo maior.

2.2 Segunda lei.

A segunda lei de Kepler será verificada tomando o produto vetorial da velocidade com a posição (em relação ao Sol), e verificando que essa quantidade é constante com o tempo. Essa grandeza é a variação da área varrida com o tempo, então o fato de ser uma constante significa que o planeta varre áreas iguais em tempos iguais, como queríamos demonstrar. Matematicamente:

$$\frac{dA}{d\tau} = \frac{|\vec{R} \times \vec{V}|}{2} = \text{constante} \iff P_x V_y - P_y V_x = \text{constante}$$

Onde na última parte dessa expressão foi assumido que os vetores posição e velocidade estão contidos no plano XY, logo o seu produto vetorial é apenas uma componente ao longo do eixo Z.

2.3 Terceira lei.

A terceira lei de Kepler será verificada tomando a razão entre o quadrado do período e o cubo do semi-eixo maior da órbita. Se essa grandeza for constante, então a terceira lei de Kepler está verificada.

O período foi calculado simulando a órbita inicialmente e contando quantas vezes o planeta intercepta o eixo X. No final das contas, o período será o dobro do tempo da última vez que o planeta passa pelo eixo X dividido pela quantidade de vezes que isso ocorreu.

Nos resultados, a verificação dessa lei para quaisquer órbitas fechadas leva ao quociente do quadrado do período pelo cubo do semi-eixo maior sendo constante e aproximadamente igual a 39,478.

Kepler's Laws ($r = 1, v = 1$)

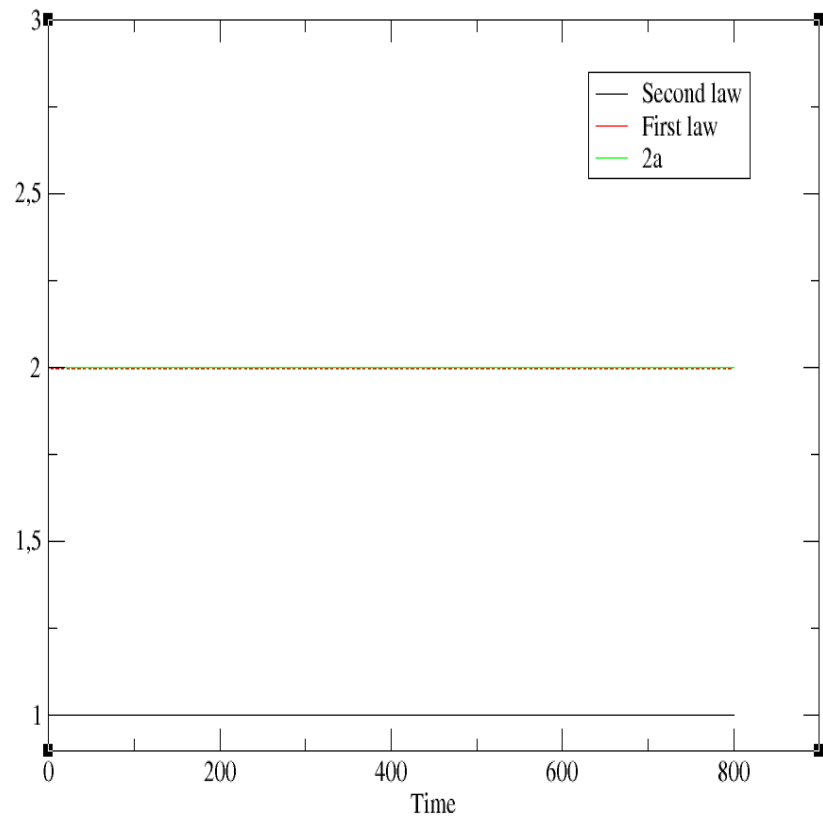


Figura 10: Verificação das leis de Kepler para uma órbita com distância inicial 1 e velocidade inicial 1.

Kepler's laws ($r = 5$, $v = 0.1$)

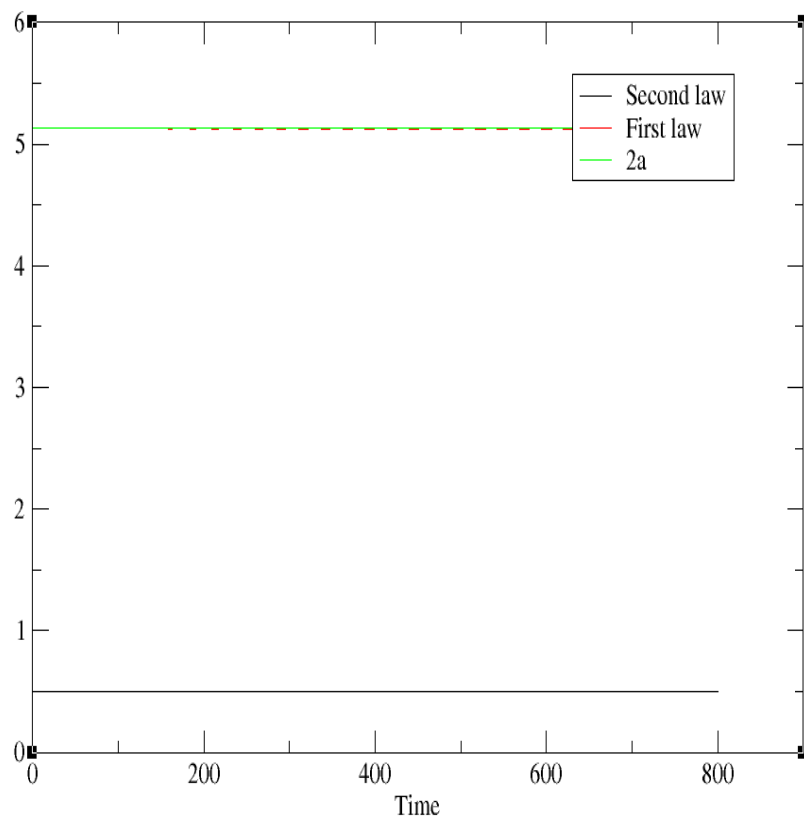


Figura 11: Verificação das leis de Kepler para uma órbita com distância inicial 5 e velocidade inicial 0,1.

Kepler's Laws (r = 0.5 , v = 1.9)

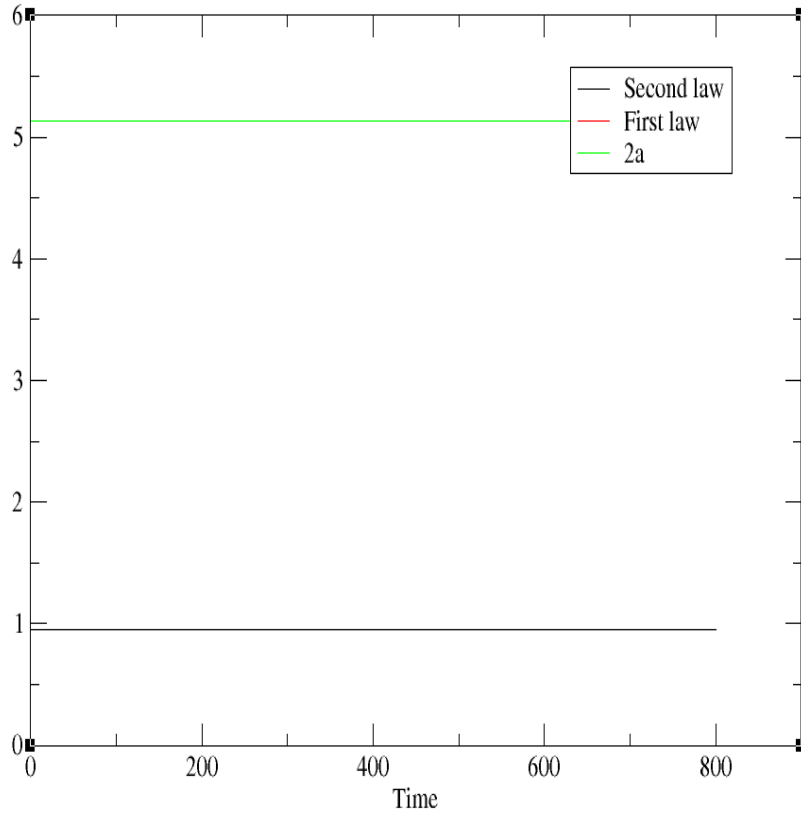


Figura 12: Verificação das leis de Kepler para uma órbita com distância inicial 0,5 e velocidade inicial 1,9.

3 Tarefa 3 - Sol, Terra e Júpiter.

Problema: Simular o movimento da Terra (e outros planetas/satélites) sob influência do Sol e Júpiter.

Pela segunda lei de Newton, as acelerações no eixo X e Y da Terra e Júpiter são, respectivamente ("T" é a Terra, "J" é o Júpiter e "S" é o Sol):

$$\begin{cases} \ddot{P}_{T,X} = -\frac{|P_{T,X}-P_{S,X}|}{|P_T-P_S|^3} - \frac{M_J}{M_S} \frac{|P_{T,X}-P_{J,X}|}{|P_T-P_J|^3} \\ \ddot{P}_{T,Y} = -\frac{|P_{T,Y}-P_{S,Y}|}{|P_T-P_S|^3} - \frac{M_J}{M_S} \frac{|P_{T,Y}-P_{J,Y}|}{|P_T-P_J|^3} \\ \ddot{P}_{J,X} = -\frac{|P_{J,X}-P_{S,X}|}{|P_J-P_S|^3} + \frac{M_T}{M_S} \frac{|P_{T,X}-P_{J,X}|}{|P_T-P_J|^3} \\ \ddot{P}_{J,Y} = -\frac{|P_{J,Y}-P_{S,Y}|}{|P_J-P_S|^3} + \frac{M_T}{M_S} \frac{|P_{T,Y}-P_{J,Y}|}{|P_T-P_J|^3} \end{cases}$$

Com isso, o movimento da Terra e do Júpiter são, pelo método de Verlet:

$$P_{i+2} = 2P_{i+1} - P_i + \ddot{P}_{\alpha,N} \Delta\tau^2$$

Onde $\ddot{P}_{\alpha,N}$ deve ser substituído pela aceleração da Terra e Júpiter nas direções X e Y (dado pelos sistemas acima). Vale a pena observar que as unidades adimensionais já foram levados em conta nessa equação.

O código fonte está nas figuras (13) e (14).

```

implicit none
real*8, parameter :: dt = 1d-3
real*8, parameter :: dt_sqrd = dt*dt
real*8, parameter :: T_max = 60*dacos(-1d0) ! 30 years
real*8, parameter :: Dist_Earth = 1d0
real*8, parameter :: Velo_Earth = 1d0/dsqrt(Dist_Earth)
real*8, parameter :: Dist_Jup = 52d-1
real*8, parameter :: Velo_Jup = 1d0/dsqrt(Dist_Jup)
real*8, parameter :: Miu1 = 318d0/333d3 ! Mass of Jup over Sun
real*8, parameter :: Miu2 = 1d0/333d3 ! Mass of Earth over Sun
real*8 :: Px0_E, Px1_E, Px2_E, Py0_E, Py1_E, Py2_E
real*8 :: Px0_J, Px1_J, Px2_J, Py0_J, Py1_J, Py2_J
real*8 :: P_E, P_EJ, P_J, T, temp1, temp2, temp3
c Suffix "E" means EARTH and "J" means JUPITER.
open(unit=1, file="tarefa-3a-saida-1.dat")
T = 0
Px0_E = Dist_Earth
Py0_E = 0
Px1_E = Px0_E
Py1_E = Py0_E + Velo_Earth*dt
Px0_J = Dist_Jup
Py0_J = 0
Px1_J = Px0_J
Py1_J = Py0_J + Velo_Jup*dt
P_EJ = norm(Px1_J - Px1_E, Py1_J - Py1_E)
P_E = norm(Px1_E, Py1_E)
P_J = norm(Px1_J, Py1_J)
do while (T .lt. T_max)
    write(1,*) Px1_E, Py1_E
    temp1 = dt_sqrd/P_E**3d0
    temp2 = dt_sqrd/P_J**3d0
    temp3 = dt_sqrd/P_EJ**3d0
    Px2_E = 2d0*Px1_E - Px0_E - (Px1_E*temp1 +
&Miu1*(Px1_E - Px1_J)*temp3)
    Py2_E = 2d0*Py1_E - Py0_E - (Py1_E*temp1 +
&Miu1*(Py1_E - Py1_J)*temp3)
    Px2_J = 2d0*Px1_J - Px0_J + (-Px1_J*temp2 +
&Miu2*(Px1_E - Px1_J)*temp3)
    Py2_J = 2d0*Py1_J - Py0_J + (-Py1_J*temp2 +
&Miu2*(Py1_E - Py1_J)*temp3)
    Px0_E = Px1_E
    Py0_E = Py1_E
    Px1_E = Px2_E
    Py1_E = Py2_E
    Px0_J = Px1_J
    Py0_J = Py1_J
    Px1_J = Px2_J
    Py1_J = Py2_J
    P_EJ = norm(Px1_J - Px1_E, Py1_J - Py1_E)
    P_E = norm(Px1_E, Py1_E)
    P_J = norm(Px1_J, Py1_J)
    T = T + dt
end do
close(1)

```

Figura 13: Código fonte da tarefa 3a (simulação das órbitas da Terra e Júpiter).

```

real*8, parameter :: Miu1 = 318d0/333d3 ! Mass of Jup over Sun
real*8, parameter :: Miu2 = 1d0/333d3 ! Mass of Earth over Sun
real*8 :: Px0_E, Px1_E, Px2_E, Py0_E, Py1_E, Py2_E
real*8 :: Px0_J, Px1_J, Px2_J, Py0_J, Py1_J, Py2_J
real*8 :: P_E, P_EJ, P_J, T, temp1, temp2, temp3
c Suffix "E" means EARTH and "J" means JUPITER.
open(unit=1, file="tarefa-3a-saida-1.dat")
T = 0
Px0_E = Dist_Earth
Py0_E = 0
Px1_E = Px0_E
Py1_E = Py0_E + Velo_Earth*dt
Px0_J = Dist_Jup
Py0_J = 0
Px1_J = Px0_J
Py1_J = Py0_J + Velo_Jup*dt
P_EJ = norm(Px1_J - Px1_E, Py1_J - Py1_E)
P_E = norm(Px1_E, Py1_E)
P_J = norm(Px1_J, Py1_J)
do while (T .lt. T_max)
    write(1,*) Px1_E, Py1_E
    temp1 = dt_sqrd/P_E**3d0
    temp2 = dt_sqrd/P_J**3d0
    temp3 = dt_sqrd/P_EJ**3d0
    Px2_E = 2d0*Px1_E - Px0_E - (Px1_E*temp1 +
&Miu1*(Px1_E - Px1_J)*temp3)
    Py2_E = 2d0*Py1_E - Py0_E - (Py1_E*temp1 +
&Miu1*(Py1_E - Py1_J)*temp3)
    Px2_J = 2d0*Px1_J - Px0_J + (-Px1_J*temp2 +
&Miu2*(Px1_E - Px1_J)*temp3)
    Py2_J = 2d0*Py1_J - Py0_J + (-Py1_J*temp2 +
&Miu2*(Py1_E - Py1_J)*temp3)
    Px0_E = Px1_E
    Py0_E = Py1_E
    Px1_E = Px2_E
    Py1_E = Py2_E
    Px0_J = Px1_J
    Py0_J = Py1_J
    Px1_J = Px2_J
    Py1_J = Py2_J
    P_EJ = norm(Px1_J - Px1_E, Py1_J - Py1_E)
    P_E = norm(Px1_E, Py1_E)
    P_J = norm(Px1_J, Py1_J)
    T = T + dt
end do
close(1)
call system("xmgrace -nxy tarefa-3a-saida-1.dat")
contains
    function norm(a,b)
        real*8 , intent(in) :: a,b
        real*8 :: norm
        norm = dsqrt(a*a + b*b)
    end function
end

```

Figura 14: Código fonte da tarefa 3a (simulação das órbitas da Terra e Júpiter).

3.1 Estabilidade

Empregando as mesmas condições iniciais do item 1a (órbitas circulares), pode ser verificado que o movimento da Terra (agora com a influência do Júpiter) não é mais periódica. A Terra passa aproximadamente 4.6721688×10^{-5} UA da sua posição anterior a cada ano. O código fonte que comprova isso e é responsável por plotar a posição da Terra durante 30 anos ($30 \times 2\pi$ nas unidades adimensionais) está representado nas figuras (15) e (16). A posição da Terra em função do tempo está na figura (17).

A razão da máxima e mínima posição atingida pela Terra nessas simulações foi aproximadamente 1.00300953. Essa razão pode ser obtida teoricamente usando o fato de que as posições máxima e mínima são dadas por, respectivamente: $R_{max} = a(1 + e)$ $R_{min} = a(1 - e)$. Logo, $\frac{R_{max}}{R_{min}} = \frac{1+e}{1-e}$. Comparando com a excentricidade real da Terra (aproximadamente 0.017), o valor calculado experimentalmente foi cerca de 3% menor.

Vale a pena notar que a escolha do passo temporal nesta tarefa se deve às contas feitas no item 1c para achar o $\Delta\tau_{max}$ da Terra (órbita com 1 UA de raio). Então o erro acumulado na simulação da órbita terrestre é da ordem de 10^{-3} , o que é negligenciável frente à influência do Júpiter. Na verdade é menor que isso, porque o tempo que o código da tarefa 1c simulava a órbita era de aproximadamente 3 vezes mais tempo que 30 anos, então nessa tarefa, foi obtido precisão maior que o esperado.

```

implicit none
real*8, parameter :: dt = 1d-3
real*8, parameter :: dt_sqrd = dt*dt
real*8, parameter :: T_max = 60*dacos(-1d0) ! 30 years
real*8, parameter :: year = 2*dacos(-1d0) ! one year
real*8, parameter :: Dist_Earth = 1d0
real*8, parameter :: Velo_Earth = 1d0/dsqrt(Dist_Earth)
real*8, parameter :: Dist_Jup = 52d-1
real*8, parameter :: Velo_Jup = 1d0/dsqrt(Dist_Jup)
real*8, parameter :: Miu1 = 318d0/333d3 ! Mass of Jup over Sun
real*8, parameter :: Miu2 = 1d0/333d3 ! Mass of Earth over Sun
real*8 :: Px0_E, Px1_E, Px2_E, Py0_E, Py1_E, Py2_E
real*8 :: Px0_J, Px1_J, Px2_J, Py0_J, Py1_J, Py2_J
real*8 :: P_E, P_EJ, P_J, T, temp1, temp2, temp3
real*8 :: drift, last_position, P_max, P_min
real*8 :: x_max, x_min, y_max, y_min
integer :: nmbr_years
c Suffix "E" means EARTH and "J" means JUPITER.
open(unit=1, file="tarefa-3b-saida-1.dat")
T = 0
Px0_E = Dist_Earth
Py0_E = 0
Px1_E = Px0_E
Py1_E = Py0_E + Velo_Earth*dt
Px0_J = Dist_Jup
Py0_J = 0
Px1_J = Px0_J
Py1_J = Py0_J + Velo_Jup*dt
P_EJ = norm(Px1_J - Px1_E, Py1_J - Py1_E)
P_E = norm(Px1_E, Py1_E)
P_J = norm(Px1_J, Py1_J)
drift = 0
last_position = P_E
nmbr_years = 0
P_max = P_E
P_min = P_E
x_max = 0
x_min = 0
y_max = 0
y_min = 0
do while (T .lt. T_max)
write(1,*) T, P_E
temp1 = dt_sqrd/P_E**3d0
temp2 = dt_sqrd/P_J**3d0
temp3 = dt_sqrd/P_EJ**3d0
Px2_E = 2d0*Px1_E - Px0_E - (Px1_E*temp1 +
&Miu1*(Px1_E - Px1_J)*temp3)
Py2_E = 2d0*Py1_E - Py0_E - (Py1_E*temp1 +
&Miu1*(Py1_E - Py1_J)*temp3)
Px2_J = 2d0*Px1_J - Px0_J + (-Px1_J*temp2 +
&Miu2*(Px1_E - Px1_J)*temp3)
Py2_J = 2d0*Py1_J - Py0_J + (-Py1_J*temp2 +
&Miu2*(Py1_E - Py1_J)*temp3)
Px0_E = Px1_E

```

Figura 15: Código fonte da tarefa 3b (simulação das órbitas da Terra e Júpiter, assim como o cálculo do afastamento terrestre e sua excentricidade).

```

&Miu1*(Py1_E - Py1_J)*temp3)
Px2_J = 2d0*Px1_J - Px0_J + (-Px1_J*temp2 +
&Miu2*(Px1_E - Px1_J)*temp3)
Py2_J = 2d0*Py1_J - Py0_J + (-Py1_J*temp2 +
&Miu2*(Py1_E - Py1_J)*temp3)
Px0_E = Px1_E
Py0_E = Py1_E
Px1_E = Px2_E
Py1_E = Py2_E
Px0_J = Px1_J
Py0_J = Py1_J
Px1_J = Px2_J
Py1_J = Py2_J
P_EJ = norm(Px1_J - Px1_E,Py1_J - Py1_E)
P_E = norm(Px1_E,Py1_E)
P_J = norm(Px1_J,Py1_J)
T = T + dt
if (dmod(T,year) .lt. dt) then ! NEW YEAR!!! :)
    drift = drift + P_E - last_position
    last_position = P_E
    nmbr_years = nmbr_years + 1
end if
if (P_E .gt. P_max) then
    P_max = P_E
end if
if (P_E .lt. P_min) then
    P_min = P_E
end if
if (Px1_E .gt. x_max) then
    x_max = Px1_E
end if
if (Px1_E .lt. x_min) then
    x_min = Px1_E
end if
if (Py1_E .gt. y_max) then
    y_max = Py1_E
end if
if (Py1_E .lt. y_min) then
    y_min = Py1_E
end if
end do
close(1)
print*, "average drift: ", drift/nmbr_years
print*, "max/min: ", P_max/P_min
print*, "eccentricity: ", dsqrt(1 - ((dabs(y_max) + dabs(y_min)
&(dabs(x_max) + dabs(x_min)))**2)
call system("xmgrace -nxy tarefa-3b-saida-1.dat")
contains
function norm(a,b)
    real*8 , intent(in) :: a,b
    real*8 :: norm
    norm = dsqrt(a*a + b*b)
end function
end

```

Figura 16: Código fonte da tarefa 3b (simulação das órbitas da Terra e Júpiter, assim como o cálculo do afastamento terrestre e sua excentricidade).

Distance of Earth from Sun (with Jupiter's influence)

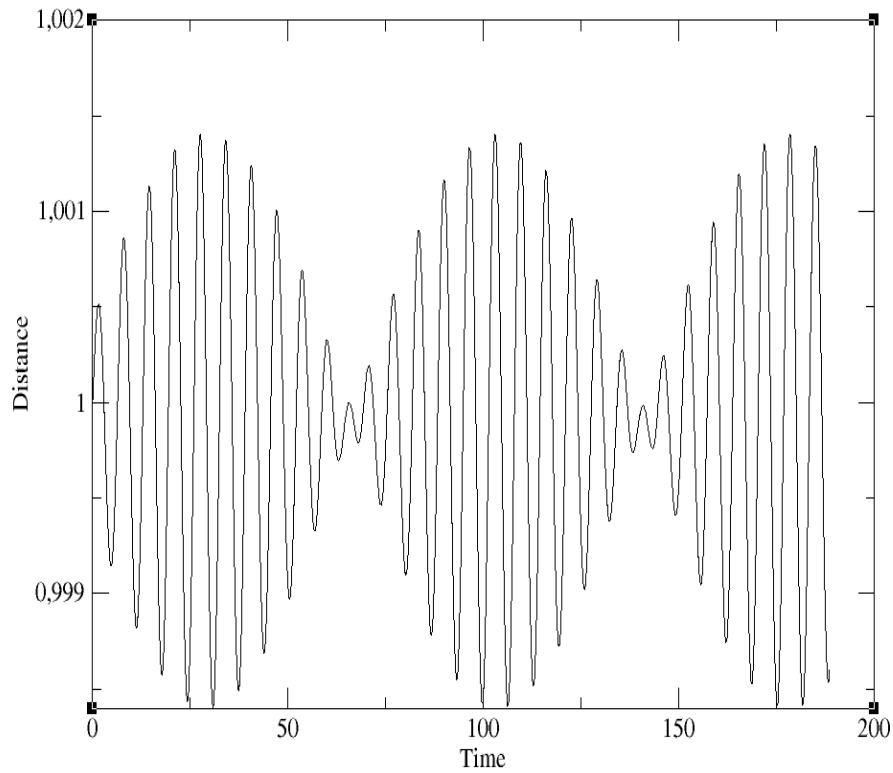


Figura 17: Distância da Terra ao Sol em função do tempo (levando em conta a influência do Júpiter).

Se a massa de Júpiter for aumentada em 100 vezes, é de se esperar que as oscilações que a Terra faz na figura (17) se exacerba , porque agora o Júpiter "puxa" a Terra com maior intensidade, deslocando-lhe ainda mais da sua órbita circular (fixa em 1 UA). Mantendo as mesmas condições iniciais da tarefa 1c, a posição da Terra e Júpiter após 30 anos está mostrada na figura (18). O código fonte é idêntica com as figuras (13) e (14) porém com a massa de Júpiter 100 vezes maior.

Position of Earth and Jupiter if Jupiter was 100x heavier

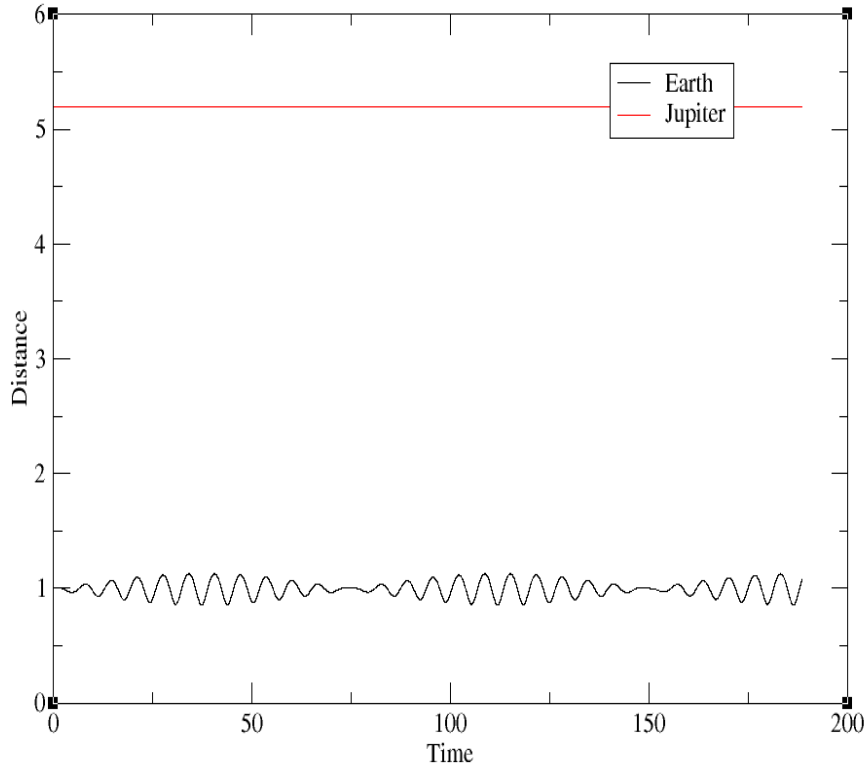


Figura 18: Distância da Terra e Júpiter ao Sol em função do tempo (Supondo que a massa de Júpiter seja cem vezes maior). Embora não parece, as oscilações da Terra são maiores que na figura (17).

Refazendo essa simulação algumas vezes, para tempos de simulação maiores, foi notado que o sistema solar imediatamente deixa de ser estável se a massa do Júpiter for cerca de mil vezes maior. Porém, se a massa do Júpiter for cem vezes maior, pode ser que deixa de ter estabilidade também, porque nesse caso, as oscilações acentuadas da Terra pode provocar forças maiores em outros planetas e luas, o que pode causar uma série de colisões em cadeia que acabaria com a vida na Terra. Essa hipótese será testada na última seção desse relatório (3.3).

3.2 Lacunas de Kirkwood

Se for dado informações da posição e velocidade de um planeta/satélite em UA e UA por ano, então estes dados se traduzem para as unidades adimensionais da seguinte forma:

$$R \text{ (UA)} \rightarrow \rho$$

e

$$V \text{ (UA/ano)} \rightarrow \frac{1}{2\pi} \frac{d\rho}{d\tau}$$

Agora será simulado a órbita de três asteróides entre marte e Júpiter. Novamente, o código fonte não está representado aqui por ser quase idêntica com as figuras (13) e (14): a única diferença reside no fato de que o asteróide agora faz o papel da Terra (apenas simbolicamente, pois na prática não mudei o nome das variáveis), e para evitar que o asteróide simulado não exerça influência no Júpiter, foi mudado a massa da Terra (nesse caso, o asteróide) para zero (também foi aumentado o tempo máximo de simulação para mil unidades adimensionais, para visualizar melhor as lacunas de Kirkwood). Então, efetivamente, é como se o asteróide sentisse a força do Sol e do Júpiter, mas estes não sentem a força do asteróide.

As órbitas dos asteróides estão respresentados nas figuras (19), (20) e (21). Pode ser notado que há espaços (lacunas de Kirkwood) entre as variações das órbitas de alguns asteróides. Na figura (22) está mostrado uma parte da órbita da figura (21) para melhor visualizar essas lacunas.

Orbit of object near Jupiter. R = 3.000 UA, V = 3.628 UA/year

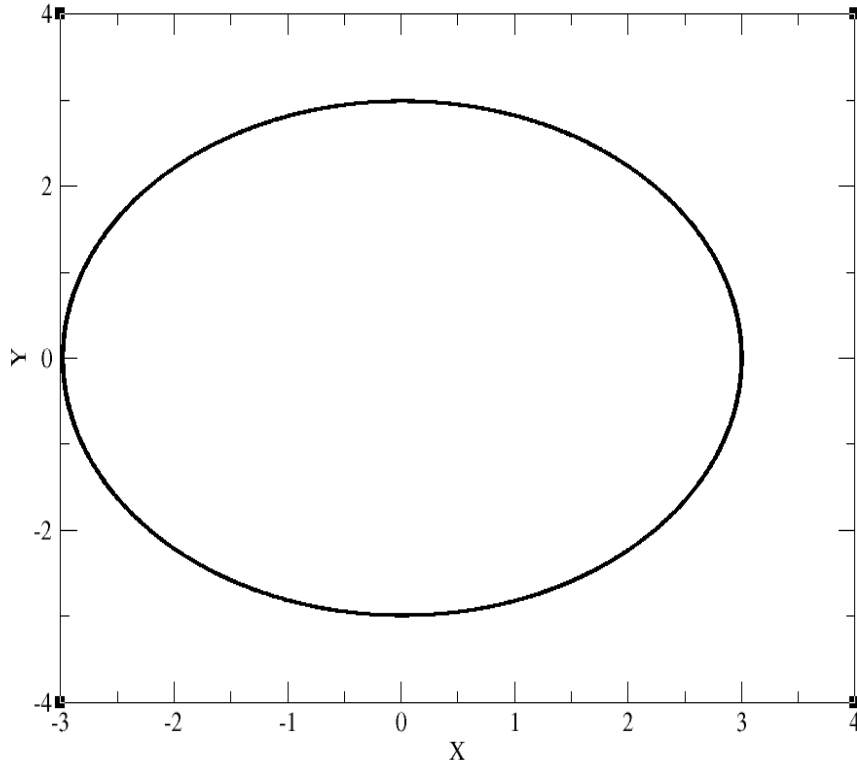


Figura 19: Órbita de um objeto entre Júpiter e marte com raio de órbita inicial de 3.000 UA e velocidade inicial de 3.628 UA por ano.

Orbit of object near Jupiter. $R = 3.276$ UA, $V = 3.471$ UA/year

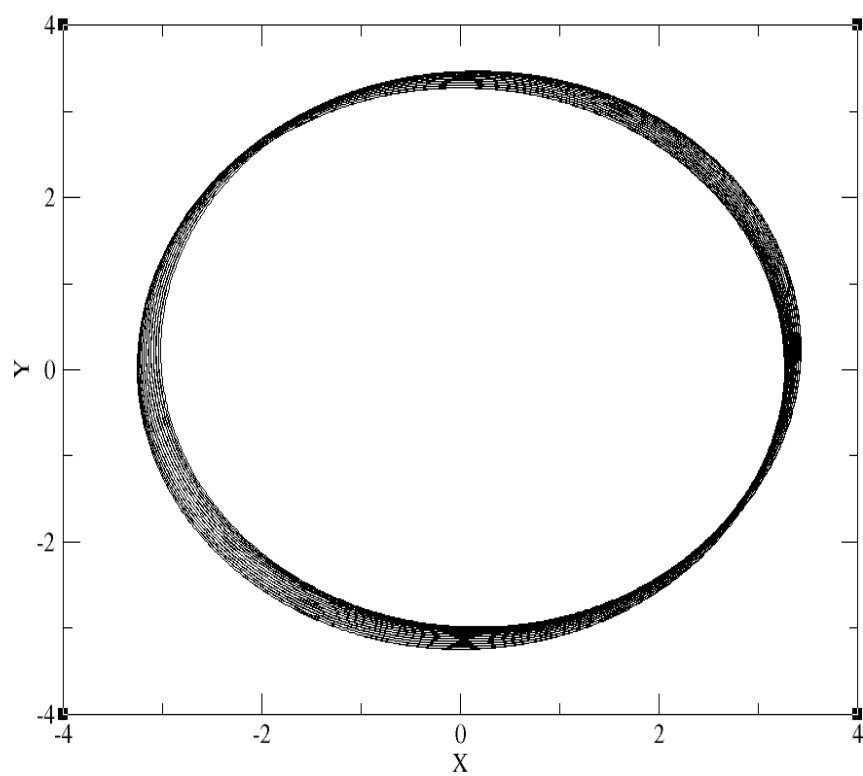


Figura 20: Órbita de um objeto entre Júpiter e marte com raio de órbita inicial de 3.276 UA e velocidade inicial de 3.471 UA por ano.

Orbit of object near Jupiter. $R = 3.700$, $V = 3.267$ UA/year

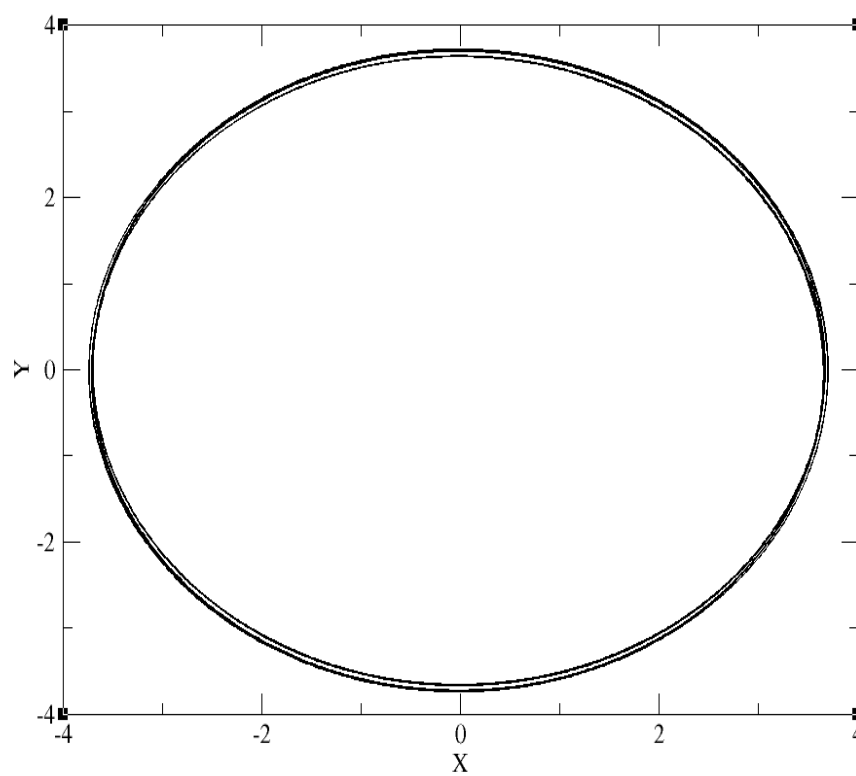


Figura 21: Órbita de um objeto entre Júpiter e marte com raio de órbita inicial de 3.700 UA e velocidade inicial de 3.267 UA por ano.

Orbit of object near Jupiter. $R = 3.700$, $V = 3.267$ UA/year

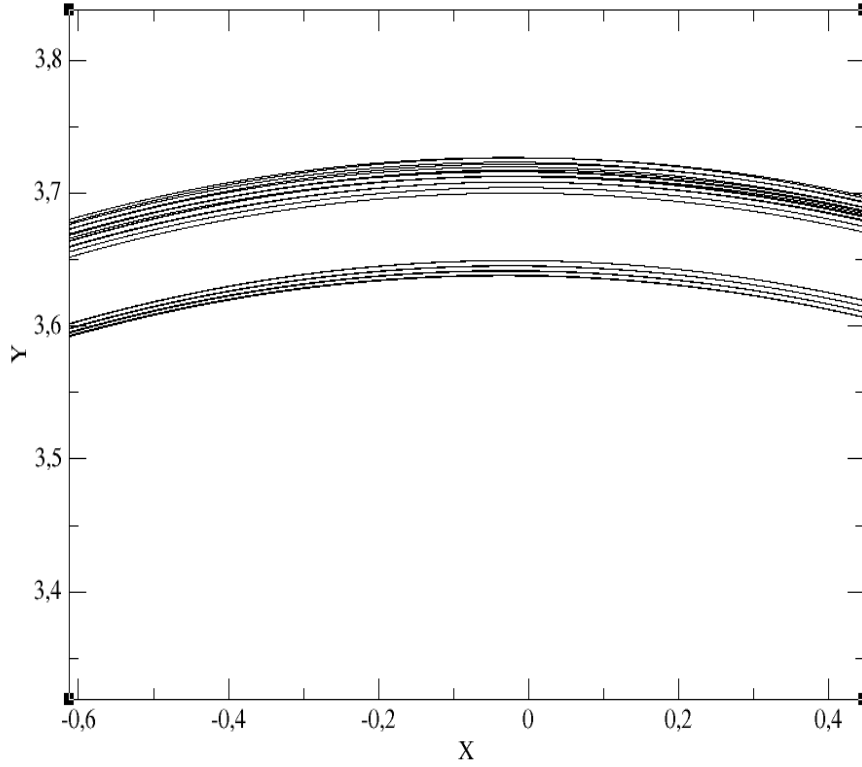


Figura 22: Parte da órbita de um objeto entre Júpiter e marte com raio de órbita inicial de 3.700 UA e velocidade inicial de 3.267 UA por ano. O vão é uma lacuna de Kirkwood.

3.3 Sistema Solar.

Por último, foi modelado todos os planetas do sistema solar com os dados fornecidos do site. Foi suposto que os planetas começam no semi-eixo maior com velocidade igual ao recíporco da raíz quadrada deste. O código fonte está nas figuras (23) e (24). O resultado está na figura (25) (foi plotado o módulo da distância de cada planeta ao sol). Na figura (26), está a mesma simulação, porém supondo que a massa de Júpiter fosse cem vezes mais pesada (há instabilidade como foi predito).

Genericamente, supondo o sol na origem e M planetas orbitando-lhe, a aceleração que cada planeta sente é dado pela segunda lei de Newton pela equação (9) a seguir:

$$\frac{d^2}{d\tau^2}(P_{N,K}) = -\frac{P_{N,K}}{P_N^3} - \sum_{i=1, i \neq N}^M \frac{M_i}{M_{sol}} \frac{(P_{N,K} - P_{i,K})}{(P_N - P_i)^3} \quad (9)$$

Onde " $P_{N,K}$ " é a posição do N -ésimo planeta na direção K (x ou y), " P_N " é

o módulo da posição do N-ésimo planeta, " M_i ", " $P_{i,K}$ " e " P_i " são as massas, posições na direção K e módulo da posição do i-ésimo planeta (que é todo outro planeta, tirando o N-ésimo planeta).

A parte difícil dessa tarefa foi implementar a equação (9) sem dor de cabeça. A solução que cheguei para isso foi generalizar o método de Verlet utilizado anteriormente. Isto é, ao invés de usar variáveis individuais para a posição de cada planeta, foi usado matrizes (o que me salvou um total de 60 variáveis a menos para serem utilizados/lembrados). Isso foi feito criando quatro matrizes: uma para guardar a razão da massa de cada planeta pela massa do Sol, uma para guardar o módulo da posição de cada planeta e duas matrizes de duas dimensões para guardar as três posições requeridas para o método de Verlet em cada direção (x ou y) para cada planeta. As matrizes guardam informações supondo uma ordenação dos planetas, então foi suposto que essa ordem será aquela de proximidade com o Sol (mercúrio é 1, Vênus é 2, ...). Então, por exemplo, se o programa acessar a matriz das massas com índice 8, então será devolvido a razão da massa de Netuno com a massa do Sol. Ou ainda, se for acessado a matriz da posição em x com índices (3,2), então será devolvido a posição x da Terra no instante médio entre os passos do método de Verlet (é o ρ_i da equação (5). ρ_{i+1} e ρ_{i-1} da Terra serão acessados pelos índices (3,3) e (3,1), respectivamente). Além disso, foi criado uma subrotina que executa um passo do método de Verlet para cada planeta e atualiza as posições para usar na próxima iteração.

```

implicit none
real*8, parameter :: dt = 1d-3
real*8, parameter :: dt_sqrd = dt*dt
real*8, parameter :: T_max = 1d2
real*8 :: M(8), Px(8,3), Py(8,3), P(8), T
integer :: i
! Masses of planets over the sun's mass
M(1) = 55d-3/333d3 ! Mercury
M(2) = 815d-3/333d3 ! Venus
M(3) = 1d0/333d3 ! Earth
M(4) = 107d-3/333d3 ! Mars
M(5) = 318d0/333d3 * 1d2 ! Jupiter (remove 1d2 for real mass)
M(6) = 952d-1/333d3 ! Saturn
M(7) = 145d-1/333d3 ! Uranus
M(8) = 171d-1/333d3 ! Neptune
! P(a,b) -> a = number of planet (mercury is 1, venus is 2, ...)
! and b = 1, 2 or 3 which are the three positions needed in the
! Verlet equation. For example, Py(4,2) is Mars' second stored
! position in the y direction.
Px(1,1) = 39d-2
Px(2,1) = 72d-2
Px(3,1) = 1d0
Px(4,1) = 152d-2
Px(5,1) = 52d-1
Px(6,1) = 958d-2
Px(7,1) = 192d-1
Px(8,1) = 301d-1
Py(:,1) = 0d0

do i = 1,8
! getting next positions using Euler's method
Px(i,2) = Px(i,1)
Py(i,2) = dt/dsqr(Px(i,1))
P(i) = norm(Px(i,2),Py(i,2)) ! array to store the norms
end do

open(unit=1, file="tarefa-3e-saida-2.dat")
T = 0
do while (T .lt. T_max)
write(1,*) T, P(1), P(2), P(3), P(4), P(5), P(6), P(7), P(8)
do i = 1,8 ! iterate once for every planet
call Verlet(Px, Py, i)
P(i) = norm(Px(i,2),Py(i,2))
end do
T = T + dt
end do
close(1)
call system("xmgrace -nxy tarefa-3e-saida-2.dat")
contains
function norm(a,b)
real*8 , intent(in) :: a,b
real*8 :: norm
norm = dsqrt(a*a + b*b)
end function

```

Figura 23: Código fonte da tarefa 3e (simulando a órbita do sistema solar).

```

Px(7,1) = 192d-1
Px(8,1) = 301d-1
Py(:, :) = 0d0

do i = 1,8
! getting next positions using Euler's method
  Px(i,2) = Px(i,1)
  Py(i,2) = dt/dsqr(Px(i,1))
  P(i) = norm(Px(i,2),Py(i,2)) ! array to store the norms
end do

open(unit=1, file="tarefa-3e-saida-2.dat")
T = 0
do while (T .lt. T_max)
  write(1,*) T, P(1), P(2), P(3), P(4), P(5), P(6), P(7), P(8)
  do i = 1,8 ! iterate once for every planet
    call Verlet(Px, Py, i)
    P(i) = norm(Px(i,2),Py(i,2))
  end do
  T = T + dt
end do
close(1)
call system("xmgrace -nxy tarefa-3e-saida-2.dat")
contains
  function norm(a,b)
    real*8 , intent(in) :: a,b
    real*8 :: norm
    norm = dsqrt(a*a + b*b)
  end function

  subroutine Verlet(Px, Py, i)
! executes one iteration of the Verlet method and preps for next
! "i" is the planet (Mercury is 1, venus is 2, ...)
    real*8 , intent(in out) :: Px(8,3), Py(8,3)
    integer , intent(in) :: i
    real*8 :: acceleration_x, acceleration_y, temp
    integer :: j
    acceleration_x = Px(i,2)/(P(i)**3)
    acceleration_y = Py(i,2)/(P(i)**3)
    do j=1,8
      if (j .ne. i) then ! have to check, else divides by zero
        temp = M(j)/(P(i) - P(j))**3
        acceleration_x = acceleration_x + temp*(Px(i,2) - Px(j,2))
        acceleration_y = acceleration_y + temp*(Py(i,2) - Py(j,2))
      end if
    end do
    Px(i,3) = 2*Px(i,2) - Px(i,1) - acceleration_x*dt_sqrd
    Py(i,3) = 2*Py(i,2) - Py(i,1) - acceleration_y*dt_sqrd
    Px(i,1) = Px(i,2)
    Px(i,2) = Px(i,3)
    Py(i,1) = Py(i,2)
    Py(i,2) = Py(i,3)
  end subroutine
end

```

Figura 24: Código fonte da tarefa 3e (simulando a órbita do sistema solar).

Orbits of planets in solar system

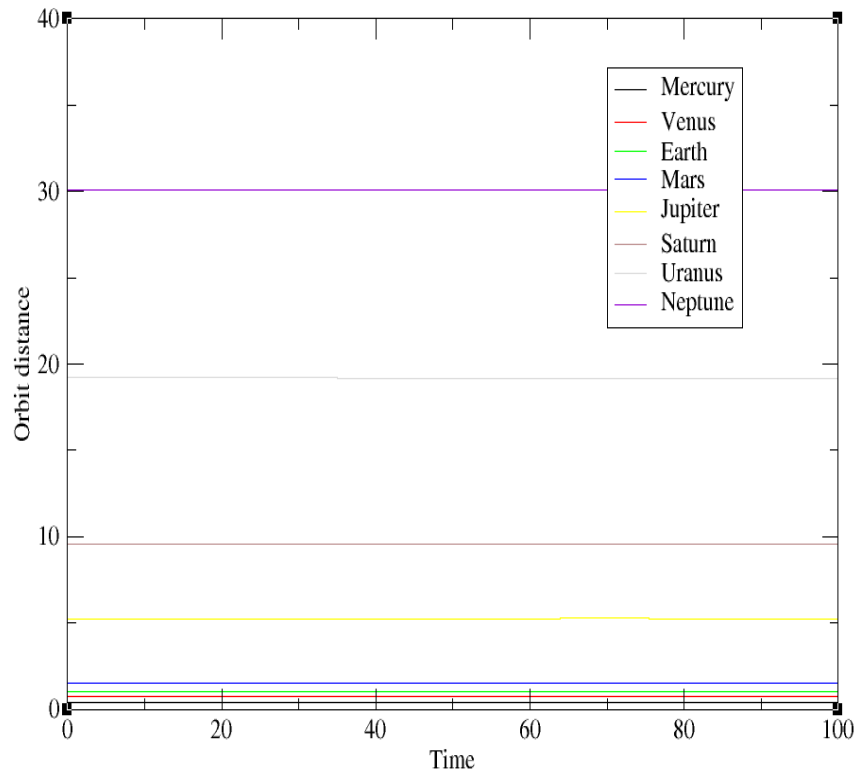


Figura 25: Órbitas dos planetas do sistema solar.

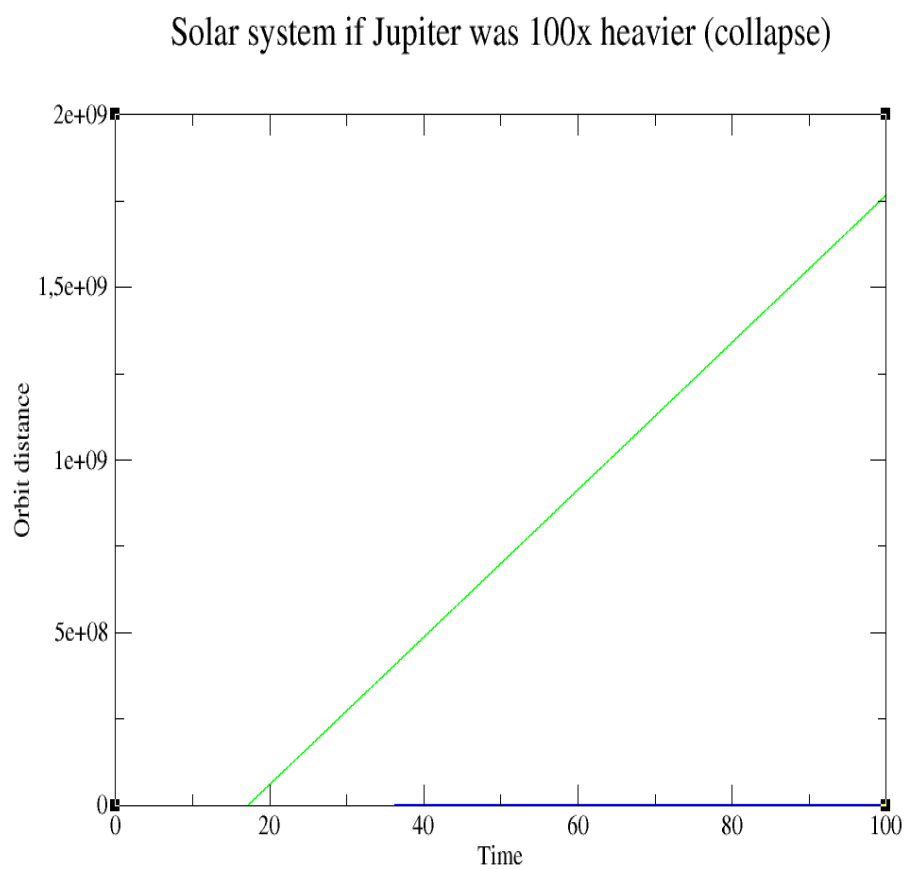


Figura 26: Órbitas dos planetas do sistema solar se o Júpiter fosse cem vezes mais pesado: há colapso do sistema solar (a hipótese estava certa!).